

PROGRAMAÇÃO FRONT-END

SEMANA 17

Integração com APIs externas, WebSockets e GraphQL

3 aulas de 50 minutos

Parte 1 - Roteiro teórico do professor
Parte 2 - Atividade prática dos estudantes
Parte 3 - Orientações, gabarito e avaliação

Material adaptado dos conteúdos da Semana 17 fornecidos para a disciplina.

PARTE 1 - ROTEIRO TEÓRICO DO PROFESSOR

Aula 1 - 50 minutos | Integração com serviços externos e comunicação em tempo real

1. Objetivos de aprendizagem

- Compreender o que é uma API e como o front-end solicita dados de serviços externos.
- Interpretar respostas em JSON e reconhecer o papel de fetch(), async/await e tratamento de erros.
- Diferenciar, em nível introdutório, HTTP tradicional e WebSocket.
- Compreender por que WebSockets são adequados para chats e outras aplicações em tempo real.
- Reconhecer os conceitos básicos de GraphQL: Query, Schema e Resolver.
- Relacionar REST, GraphQL e WebSocket a necessidades diferentes de integração.

2. Conhecimentos prévios

Espera-se que os estudantes já reconheçam a estrutura básica de HTML, CSS e JavaScript, saibam manipular elementos simples do DOM e tenham contato inicial com funções em JavaScript.

3. Recursos

- Computador do professor e projetor.
- Lousa e marcador.
- Laboratório de informática para as aulas práticas.
- Node.js disponível nas máquinas para a prática, quando possível.
- Navegador e editor de código.

4. Cronograma da aula teórica

Tempo	Tema	Objetivo
5 min	Integração entre sistemas	Contextualizar APIs com exemplos do cotidiano.
15 min	APIs, JSON e fetch()	Entender requisição, resposta, dados e tratamento de erros.
15 min	WebSockets	Compreender comunicação persistente e em tempo real.
10 min	GraphQL	Introduzir Query, Schema e Resolver.
5 min	Síntese	Comparar tecnologias e preparar a prática.

5. Ponto de partida: de onde vêm os dados?

Pergunta para iniciar: “Um aplicativo de clima possui dentro dele a temperatura atual de todas as cidades?”

Condução: deixe os estudantes levantarem hipóteses e, em seguida, explique que aplicações frequentemente dependem de serviços externos para obter informações atualizadas.

Esquema para a lousa

```
FRONT-END
| requisição
v
API
| resposta
v
FRONT-END
```

6. APIs, requisição e resposta

API (Application Programming Interface) é uma interface que permite a comunicação entre aplicações. No contexto desta aula, o front-end realiza uma requisição para um serviço e recebe uma resposta com os dados disponíveis.

Termo	Explicação simples
Requisição	Pedido enviado pelo cliente para um serviço.
Resposta	Retorno enviado pelo serviço.
Endpoint	Endereço usado para acessar uma operação ou conjunto de dados da API.
JSON	Formato de texto muito utilizado para transportar dados.
Status HTTP	Código que ajuda a indicar o resultado da requisição.

Exemplos de status úteis para a aula: 200 = sucesso; 404 = recurso não encontrado; 500 = erro no servidor.

7. JSON na prática

Exemplo de resposta JSON

```
{
  "cidade": "São Paulo",
  "temperatura": 24,
  "condicao": "Nublado"
}
```

Acessando propriedades em JavaScript

```
console.log(dados.cidade);
console.log(dados.temperatura);
```

Explique que, após transformar uma resposta JSON em objeto JavaScript, os campos podem ser acessados por suas propriedades.

8. fetch(), async/await e erros

Exemplo didático

```
async function buscarDados() {
  try {
    const resposta = await fetch("URL_DA_API");

    if (!resposta.ok) {
      throw new Error("Erro ao buscar os dados");
    }

    const dados = await resposta.json();
    console.log(dados);
  } catch (erro) {
    console.error(erro);
  }
}
```

Elemento	Função
async	Indica que a função trabalha com operações assíncronas.
await	Aguarda a conclusão de uma operação assíncrona dentro da função.
fetch()	Realiza a requisição HTTP.
resposta.json()	Interpreta a resposta no formato JSON.

try/catch	Permite tratar falhas de rede, respostas inválidas ou outros erros.
-----------	---

Atenção

Não é necessário aprofundar Promises nesta aula. O foco é compreender o fluxo: solicitar, aguardar, interpretar e tratar erro.

9. Fetch x Axios

Fetch	Axios
Recurso nativo dos navegadores modernos.	Biblioteca externa.
Não exige instalação.	Precisa ser importado/instalado.
Erros HTTP precisam ser verificados manualmente com <code>response.ok</code> .	Facilita o tratamento de respostas HTTP com erro.

A prática desta sequência não depende de Axios. Ele aparece apenas para comparação, conforme o conteúdo da semana.

10. Chaves de API

Alguns serviços externos exigem uma chave para identificar e autorizar o uso da API. Em materiais didáticos, utilize sempre um marcador em vez de expor uma chave real.

```
const apiKey = "SUA_CHAVE_AQUI";
```

Boa prática

Chaves privadas não devem ser publicadas em repositórios públicos ou inseridas diretamente em exemplos distribuídos aos estudantes.

11. WebSocket: por que usar em um chat?

No HTTP tradicional, a comunicação costuma seguir o fluxo de requisição e resposta. Em WebSocket, uma conexão é estabelecida e permanece aberta, permitindo troca contínua de mensagens enquanto a conexão estiver ativa.

Comparação visual

```
HTTP
Cliente -> requisição -> Servidor -> resposta

WebSocket
Cliente <=====> Servidor
conexão persistente
```

HTTP	WebSocket
Baseado principalmente em requisição e resposta.	Comunicação contínua após a conexão.
O cliente normalmente inicia uma nova requisição para obter dados.	Cliente e servidor podem trocar mensagens durante a conexão.
Muito utilizado em APIs tradicionais.	Muito útil em chats, notificações, jogos e acompanhamento em tempo real.
Não é substituído por WebSocket.	Resolve necessidades diferentes do HTTP.

12. Node.js e a biblioteca ws

Node.js permite executar JavaScript fora do navegador. Na atividade, ele será utilizado para executar o servidor WebSocket. A biblioteca ws fornece uma implementação de WebSocket para Node.js; no navegador, o cliente usa o objeto WebSocket nativo.

```
const WebSocket = require("ws");
```

13. Eventos principais

No servidor	No cliente
connection - um cliente se conectou.	onopen - conexão aberta.
message - uma mensagem chegou.	onmessage - mensagem recebida.
close - cliente desconectou.	onclose - conexão encerrada.
	onerror - erro de comunicação.

14. Introdução ao GraphQL

GraphQL é uma linguagem de consulta utilizada em APIs que permite ao cliente declarar quais campos deseja receber. Nesta aula, o objetivo é apenas compreender o princípio.

Exemplo de Query GraphQL

```
query {  
  produto {  
    nome  
    preco  
  }  
}
```

Conceito	Explicação
Schema	Define os tipos e operações disponíveis na API.
Query	Solicita dados.
Resolver	Código que localiza e retorna os dados pedidos.

15. REST x GraphQL

REST costuma organizar recursos em endpoints, enquanto GraphQL normalmente permite formular uma consulta declarando os campos desejados. Uma abordagem não é sempre melhor que a outra; cada uma atende contextos diferentes.

Exemplo conceitual de endpoints REST

```
/api/clientes  
/api/produtos  
/api/pedidos
```

16. O que colocar na lousa

API = comunicação entre aplicações
JSON = formato de dados
fetch() = requisição HTTP no JavaScript
async/await = operações assíncronas

HTTP: Cliente -> Servidor -> Resposta
WebSocket: Cliente <-> Servidor

GraphQL = consulta os campos necessários
Schema = estrutura disponível
Query = consulta
Resolver = retorna os dados

17. Perguntas rápidas para verificar compreensão

- O que uma API permite que uma aplicação faça?
- Qual é o papel de JSON numa integração?
- Para que serve `fetch()`?
- Por que um chat combina com WebSocket?
- WebSocket substitui HTTP?
- Em GraphQL, o que uma Query faz?

18. Respostas esperadas

- Permite que aplicações troquem dados ou acessem funcionalidades de outros sistemas.
- JSON é um formato usado para transportar e organizar dados.
- `fetch()` realiza requisições HTTP no JavaScript.
- Porque o chat precisa de troca contínua de mensagens em tempo real.
- Não. São tecnologias usadas para necessidades diferentes.
- Uma Query solicita dados definidos pelo Schema e atendidos pelos Resolvers.

PARTE 2 - ROTEIRO DA ATIVIDADE PRÁTICA

Aulas 2 e 3 - 100 minutos | Criando um aplicativo de chat com WebSockets

1. Identificação

Nome(s)	Turma	Data

2. Situação-problema

Uma equipe precisa criar uma interface de chat em tempo real. O objetivo é permitir que duas ou mais pessoas conectadas ao mesmo servidor troquem mensagens imediatamente. Para isso, vocês utilizarão HTML, CSS e JavaScript no cliente, além de Node.js e WebSocket no servidor.

3. Objetivos da atividade

- Criar a estrutura de um projeto front-end simples.
- Executar um servidor WebSocket com Node.js.
- Conectar a página do navegador ao servidor.
- Enviar e receber mensagens em tempo real.
- Testar a aplicação em duas abas ou navegadores.
- Registrar evidências do funcionamento.

4. Estrutura do projeto

```
chat-app/  
|  
|-- server.js  
|-- index.html  
|-- style.css  
|-- script.js
```

AULA 2 - PARTE 1 - 50 MINUTOS

Tempo	Etapa
5 min	Criar a pasta e os arquivos.
10 min	Preparar o projeto Node.js.
15 min	Criar e compreender server.js.
15 min	Criar index.html e style.css.
5 min	Fazer o primeiro teste.

5. Preparando o projeto Node.js

1. Abra o terminal dentro da pasta chat-app.
2. Crie o package.json:

```
npm init -y
```

3. Instale a biblioteca ws:

```
npm install ws
```

Se aparecer "Cannot find module ws"

Verifique se o comando npm install ws terminou corretamente dentro da pasta do projeto.

6. Criando o servidor - server.js

Digite o código abaixo. Ele cria o servidor na porta 8081 e retransmite cada mensagem recebida para todos os clientes conectados.

```
const WebSocket = require("ws");

const servidor = new WebSocket.Server({
  port: 8081
});

servidor.on("connection", (cliente) => {
  console.log("Novo cliente conectado");

  cliente.on("message", (mensagem) => {
    const texto = mensagem.toString();

    servidor.clients.forEach((outroCliente) => {
      if (outroCliente.readyState === WebSocket.OPEN) {
        outroCliente.send(texto);
      }
    });
  });

  cliente.on("close", () => {
    console.log("Cliente desconectado");
  });
});

console.log("Servidor WebSocket executando na porta 8081");
```

Pontos importantes:

- connection é acionado quando um cliente se conecta.
- message recebe uma mensagem enviada por um cliente.
- mensagem.toString() garante que o conteúdo seja tratado como texto.
- servidor.clients contém os clientes conectados.
- readyState === WebSocket.OPEN verifica se o cliente está pronto para receber a mensagem.
- close registra a desconexão.

7. Criando a interface - index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Chat em Tempo Real</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="chat">
    <h1>Chat em Tempo Real</h1>
    <p id="status">Status: conectando...</p>
    <label for="name">Nome</label>
    <input id="name" type="text" placeholder="Digite seu nome">
    <label for="message">Mensagem</label>
    <div class="linha-envio">
      <input id="message" type="text" placeholder="Digite sua mensagem">
      <button id="sendButton" type="button">Enviar</button>
    </div>
    <section id="messages" aria-live="polite" aria-label="Mensagens do chat"></section>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

A página possui campos para nome e mensagem, um botão de envio, um indicador de status e uma área para exibir as mensagens recebidas.

8. Estilização - style.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  display: grid;
  place-items: center;
  background: #eef2f7;
  font-family: Arial, sans-serif;
}

.chat {
  width: min(92%, 560px);
  padding: 24px;
  background: #fff;
  border-radius: 12px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
}

.chat h1 {
  margin-top: 0;
}

label {
  display: block;
  margin-top: 12px;
  margin-bottom: 6px;
  font-weight: 700;
}

input,
button {
  font: inherit;
}

input {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid #aab4c0;
  border-radius: 8px;
}

input:focus {
  outline: 3px solid rgba(0, 95, 204, 0.22);
  border-color: #005fcc;
}

.linha-envio {
  display: flex;
  gap: 8px;
}

button {
  padding: 10px 16px;
  border: 0;
  border-radius: 8px;
  background: #005fcc;
  color: #fff;
  cursor: pointer;
}

#messages {
  margin-top: 18px;
  min-height: 160px;
  max-height: 280px;
  overflow-y: auto;
  padding: 12px;
  border: 1px solid #d7dde5;
  border-radius: 8px;
}
```

```
background: #f8fafc;
}

#messages p {
margin: 6px 0;
}
```

O CSS foi mantido simples. O foco principal é comunicação em tempo real, não design avançado.

9. Primeiro teste da Aula 2

4. No terminal, execute:

```
node server.js
```

5. Confirme se aparece a mensagem:

Servidor WebSocket executando na porta 8081

6. Mantenha o terminal aberto. O servidor precisa continuar executando durante o teste.

AULA 3 - PARTE 2 - 50 MINUTOS

Tempo	Etapas
10 min	Criar/finalizar script.js.
10 min	Abrir o cliente e conectar ao servidor.
10 min	Testar com duas abas ou navegadores.
10 min	Identificar e corrigir problemas.
5 min	Realizar um pequeno aprimoramento.
5 min	Registrar evidências e responder às perguntas.

10. JavaScript do cliente - script.js

```
const socket = new WebSocket("ws://localhost:8081");
const statusEl = document.getElementById("status");
const messagesEl = document.getElementById("messages");
const nameEl = document.getElementById("name");
const messageEl = document.getElementById("message");
const sendButton = document.getElementById("sendButton");

socket.onopen = () => {
  statusEl.textContent = "Status: conectado";
};

socket.onmessage = (event) => {
  const item = document.createElement("p");
  item.textContent = event.data;
  messagesEl.appendChild(item);
};

socket.onerror = () => {
  statusEl.textContent = "Status: erro de conexão";
};

socket.onclose = () => {
  statusEl.textContent = "Status: desconectado";
};

function enviarMensagem() {
  const nome = nameEl.value.trim();
  const mensagem = messageEl.value.trim();

  if (!nome || !mensagem) {
    alert("Preencha seu nome e a mensagem.");
    return;
  }
}
```

```

if (socket.readyState !== WebSocket.OPEN) {
  alert("A conexão com o servidor ainda não está pronta.");
  return;
}

socket.send(`${nome}: ${mensagem}`);
messageEl.value = "";
messageEl.focus();
}

sendButton.addEventListener("click", enviarMensagem);
messageEl.addEventListener("keydown", (event) => {
  if (event.key === "Enter") {
    enviarMensagem();
  }
});

```

O script cria a conexão, atualiza o status, recebe mensagens e envia o texto digitado pelo usuário. Ele também impede o envio quando nome ou mensagem estão vazios ou quando a conexão ainda não está aberta.

11. Como testar com dois clientes

7. Mantenha node server.js em execução.
8. Abra a página do chat em uma primeira aba ou navegador.
9. Abra a mesma página em uma segunda aba ou navegador.
10. No primeiro cliente, use um nome, por exemplo: Pedro.
11. No segundo cliente, use outro nome, por exemplo: Ana.
12. Envie uma mensagem em cada cliente.
13. Verifique se as mensagens aparecem nas duas instâncias.

12. Pequeno desafio

Escolha apenas uma melhoria e implemente-a:

- Alterar a aparência das mensagens.
- Modificar o texto de status da conexão.
- Adicionar uma pequena identificação visual para o botão de envio.

13. Conectando os conceitos

Se o chat precisasse mostrar a previsão do tempo da cidade do usuário, qual tecnologia estudada poderia buscar esses dados?

Api de clima, como a "Weather API".

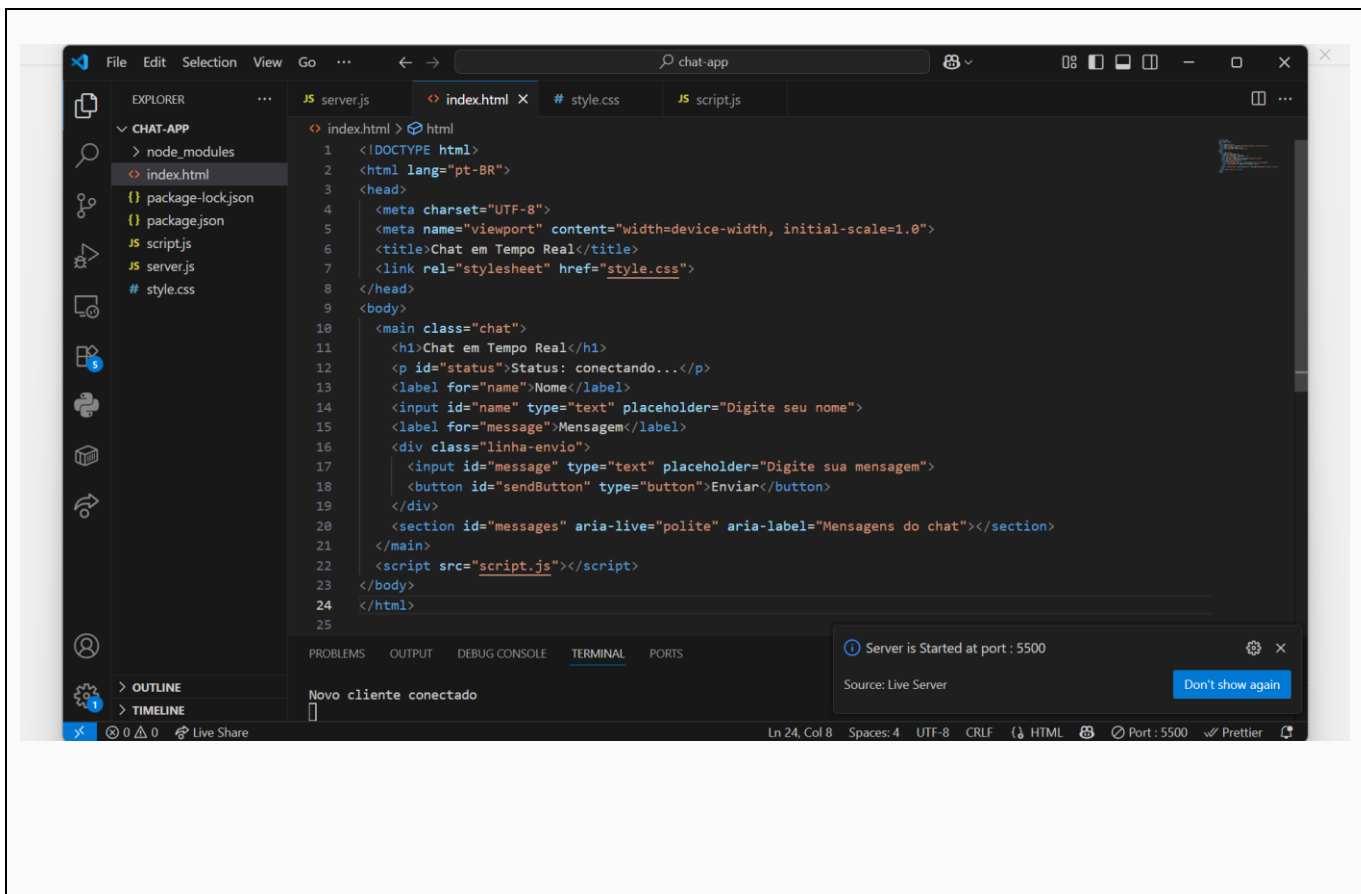
Se uma API permitisse escolher exatamente quais campos devem ser retornados, qual tecnologia estudada poderia ser utilizada?

A tecnologia que poderia ser utilizada é o GraphQL, pois permite escolher exatamente quais campos serão retornados pela API.

14. Evidências da atividade

EVIDÊNCIA 1 - Estrutura do projeto

Insira um print do editor mostrando server.js, index.html, style.css e script.js.

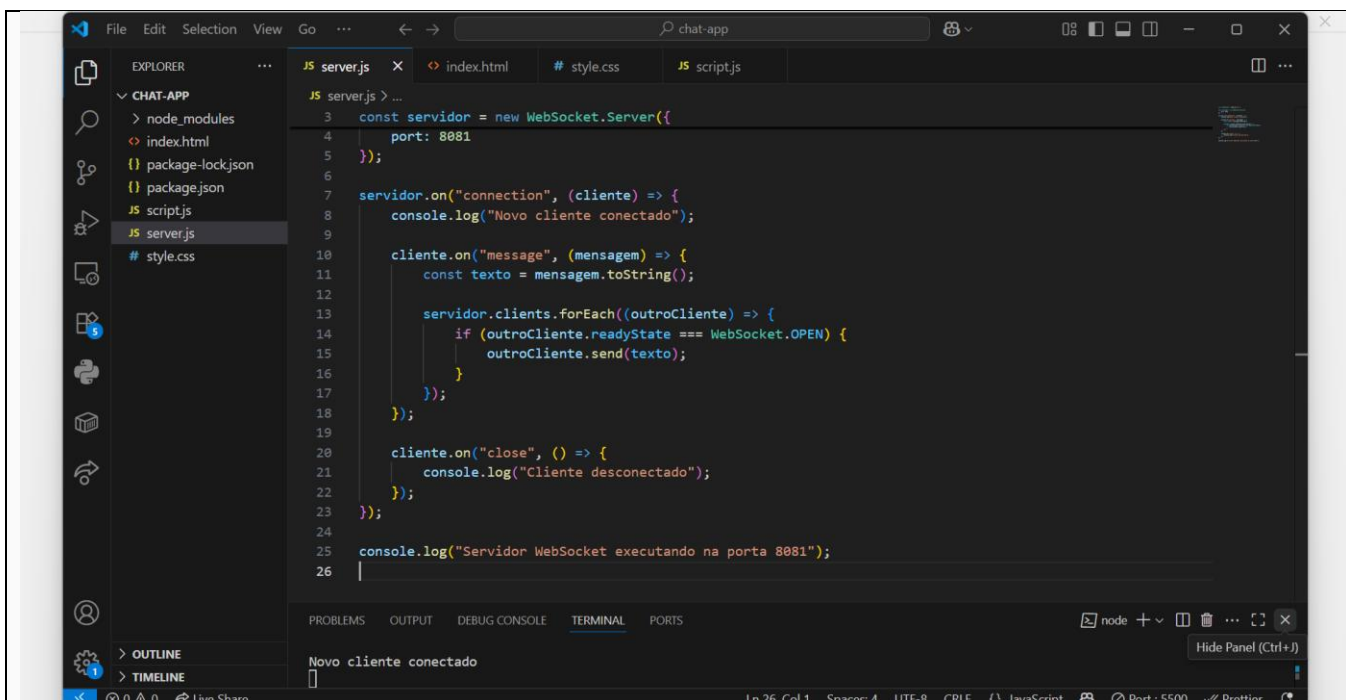


Descrição breve da evidência:

Todos os arquivos criados e executados conforme pedido.

EVIDÊNCIA 2 - Servidor executando

Insira um print do terminal mostrando o servidor WebSocket ativo.

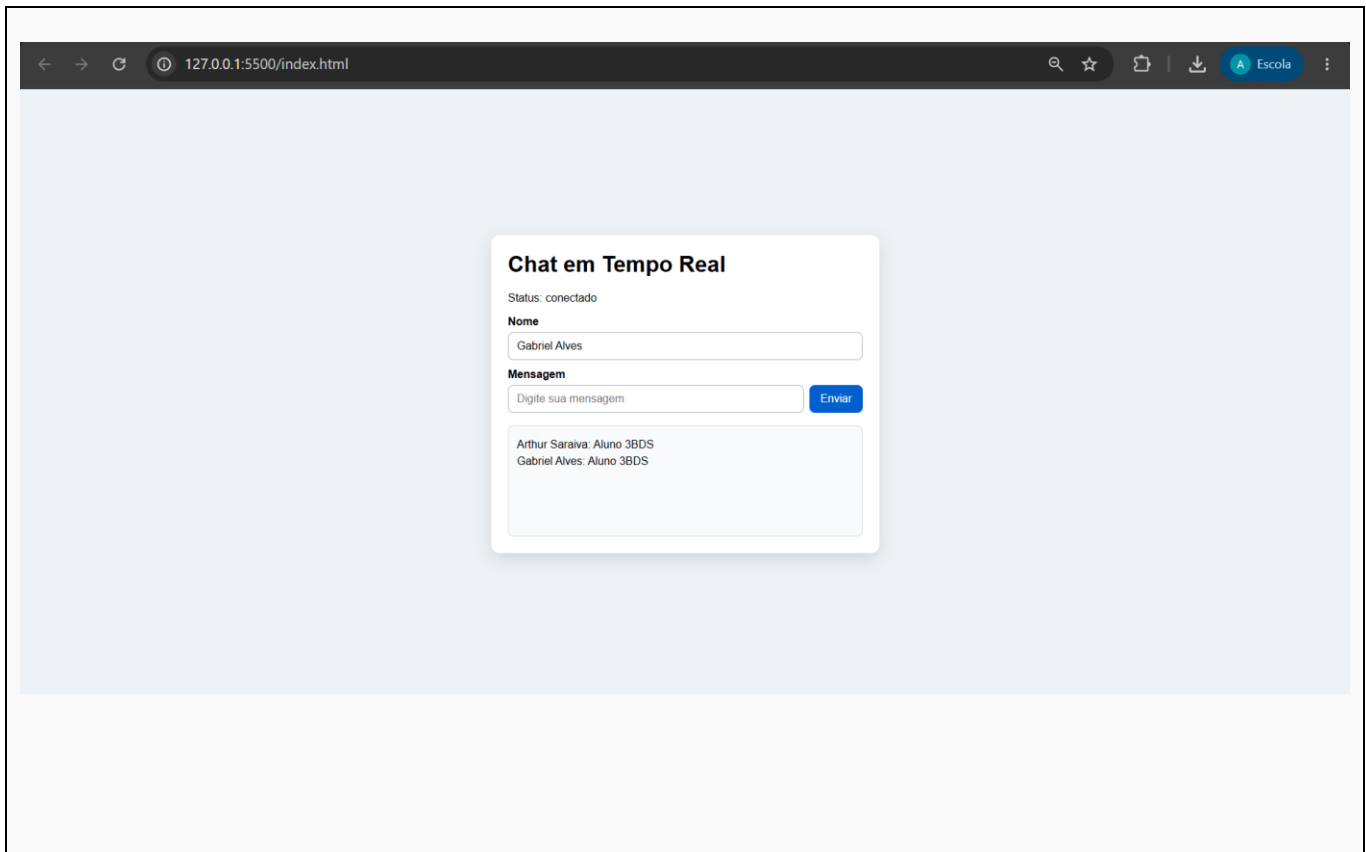


Descrição breve da evidência:

Funcionando corretamente.

EVIDÊNCIA 3 - Chat funcionando

Insira um print mostrando mensagens exibidas na interface.

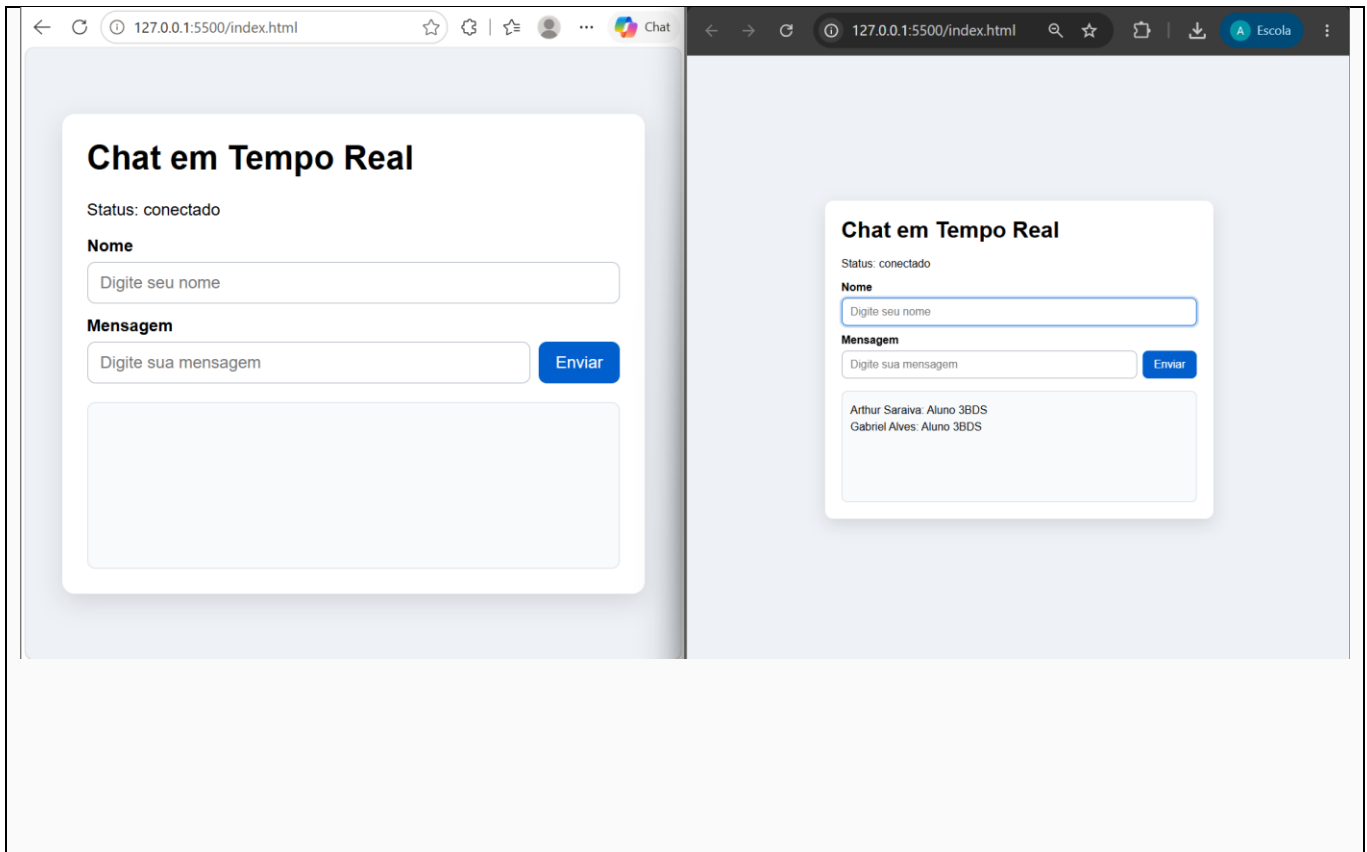


Descrição breve da evidência:

Mensagens com os nomes dos usuários.

EVIDÊNCIA 4 - Dois clientes

Insira um print que demonstre a comunicação em duas abas ou navegadores.



Descrição breve da evidência:

Imagem do chatbot em dois navegadores.

15. Perguntas finais

1. O que é uma API?

É uma forma de comunicação entre diferentes sistemas, permitindo que um programa acesse dados ou funções de outro programa.

2. Para que serve fetch() no JavaScript?

Serve para fazer requisições a APIs e receber ou enviar dados pela internet.

3. Qual é a diferença básica entre HTTP tradicional e WebSocket?

HTTP funciona por requisições e respostas, enquanto WebSocket mantém uma conexão aberta para comunicação em tempo real.

4. Por que WebSocket é adequado para um aplicativo de chat?

Porque permite enviar e receber mensagens em tempo real, sem precisar fazer novas requisições a todo momento.

5. Para que serve GraphQL em uma integração com API?

Serve para buscar exatamente os dados necessários da API, evitando receber informações desnecessárias.

16. Checklist antes de entregar

- ☐ Criei os quatro arquivos do projeto.
- ☐ Preparei o projeto Node.js.
- ☐ Instalei a biblioteca ws.
- ☐ Criei o servidor WebSocket.

- ☐ Criei a interface HTML.
- ☐ Apliquei o CSS.
- ☐ Criei o JavaScript do cliente.
- ☐ Conectei o cliente ao servidor.
- ☐ Enviei e recebi mensagens.
- ☐ Testei com dois clientes.
- ☐ Registre os quatro prints.
- ☐ Respondi às perguntas finais.

17. Critérios de avaliação

Critério	Valor
Estrutura HTML	1,0
CSS e organização	1,0
Servidor WebSocket	2,0
JavaScript do cliente	2,0
Comunicação em tempo real	2,0
Testes e evidências	1,0
Perguntas finais e organização	1,0
TOTAL	10,0